

CO4 : ASSESSMENT TOOL 3: Reverse Engineering Task

SET 1

1. **A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.**

ANSWER:

1. Large-Scale Website, Mobile, and Transaction Data Platform

A big data platform processing billions of clicks, mobile interactions, and customer transactions would most likely use a distributed storage architecture such as Hadoop Distributed File System (HDFS) or cloud object storage such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. These systems divide data across multiple nodes, allowing very large datasets to be stored reliably and processed in parallel.

For data ingestion, a distributed framework such as Apache Kafka would probably collect website clicks, mobile events, and transaction records in real time. Batch ingestion could use tools such as Apache Sqoop or ETL pipelines. The collected data would typically be stored in a data lake, with raw clickstream and log data stored in formats such as JSON, Avro, or Parquet.

Structured data such as customer profiles, orders, payments, and product information would generally be stored in relational or distributed SQL databases. Semi-structured data such as clickstream events, application logs, and mobile events would be stored in the data lake. Partitioning by date, region, customer segment, or event type would make queries faster.

Columnar formats such as Parquet could further improve analytical performance.

For batch processing, Apache Spark or Hadoop MapReduce would likely process historical data. Real-time analytics could use Apache Flink, Spark Streaming, or Kafka Streams. These engines can process incoming events continuously and generate metrics such as active users, conversion rates, and transaction trends.

Scalability would be achieved through horizontal scaling, where additional servers are added when workload increases. Kafka topics could be divided into partitions, while storage and Spark workloads could be distributed across multiple nodes. Replication would provide fault tolerance if a server fails.

Finally, dashboards and business intelligence systems such as Power BI, Tableau, or Looker would connect to the processed analytical layer. Precomputed KPIs and aggregated datasets could be exposed through SQL query engines such as Presto/Trino, Hive, or cloud data warehouses, allowing managers to monitor sales, customer behavior, traffic, and business performance.

2. **An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.**

ANSWER:

E-Commerce Personalization and Recommendation Platform:

An e-commerce platform tracking product views, cart additions, abandoned checkouts, and payment behavior would probably use Apache Kafka as the central event-streaming platform. Each event could be published to separate Kafka topics such as `product_views`, `cart_events`, `checkout_events`, and `payments`.

Customer profiles would likely be maintained in a combination of relational databases and distributed NoSQL databases. A system such as Apache Cassandra, MongoDB, DynamoDB, or Redis could maintain frequently accessed customer and session information. Redis would be particularly useful for low-latency access to active sessions, recommendations, and customer preferences.

Clickstream logs and session events would be collected continuously. Kafka would synchronize these events between applications and downstream systems. Transaction information from the order database could be joined with browsing information in the analytical layer. This allows the system to determine relationships such as:

Customer → Viewed Product → Added to Cart → Abandoned Cart → Purchased Product

A recommendation workflow would typically contain several stages. First, raw events are collected. Second, customer features such as frequently viewed categories, purchase frequency, and recent activity are generated. Third, machine learning models such as collaborative filtering, content-based recommendation, matrix factorization, or deep-learning recommendation models generate product recommendations. Finally, recommendations are cached in Redis or another low-latency store so that they can be delivered immediately.

Distributed query engines such as Trino/Presto, Spark SQL, or Hive could analyze large historical datasets. Structured order data can be joined with semi-structured browsing data using customer IDs, session IDs, product IDs, and timestamps.

Fault tolerance would be provided through Kafka replication, replicated databases, distributed storage, and checkpointing in stream-processing frameworks. Partitioning could be based on customer ID, region, product category, or event timestamp.

Business dashboards would calculate KPIs such as conversion rate, cart abandonment rate, average order value, revenue by region, customer lifetime value, and recommendation click-through rate. These metrics could be visualized through Power BI, Tableau, Looker, or similar BI tools.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

ANSWER:

Big Data Healthcare Platform:

A healthcare big data system handling patient histories, diagnostic images, wearable data, and prescriptions would require a hybrid storage architecture because the data has different structures and sensitivity levels.

Structured information such as patient demographics, prescriptions, laboratory results, appointments, and billing information would typically be stored in relational databases such as PostgreSQL, MySQL, or distributed SQL databases. Semi-structured information such as electronic health record messages and clinical documents could be stored using document databases or data-lake storage. Large diagnostic images such as X-rays, CT scans, and MRI images would generally be stored in object storage or specialized medical-image systems.

Data integration between hospitals, laboratories, pharmacies, and insurance providers could use HL7 and FHIR interoperability standards. ETL/ELT tools and healthcare integration platforms would extract data from different systems, transform it into common formats, and load it into the analytical platform.

Wearable devices generate continuous sensor streams such as heart rate, oxygen saturation, temperature, and activity. Kafka, Apache Flink, or Spark Streaming could process these streams. Real-time analytics could identify abnormal patterns and generate alerts.

Machine learning models could be developed using Apache Spark MLlib, Python, TensorFlow, or PyTorch. Historical patient data could be used for disease-risk prediction, readmission prediction, patient deterioration detection, and population-health analysis. Medical images could additionally be processed using deep-learning models such as convolutional neural networks.

Because healthcare data is highly sensitive, the platform would require strong encryption, authentication, authorization, auditing, and access control. Data should be encrypted both at rest and during transmission. Role-based access control would ensure that doctors, nurses, administrators, researchers, and insurance personnel only access the information required for their roles. Audit logs would record who accessed or modified patient information.

The analytical pipeline would therefore be:

Healthcare Sources → Secure Ingestion → Data Lake/Databases → Cleaning & Integration → Distributed Processing → ML Analytics → Clinical Decision Support/Dashboards

The final analytics layer could support both individual patient care and large-scale population-health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

ANSWER:

Smart City Big Data Platform:

A smart city platform receiving traffic sensors, CCTV metadata, weather information, and public transport data would probably use edge computing to process some information close to where it is generated. Traffic cameras and sensors could perform initial filtering at edge devices instead of sending every raw event to a central data center.

A distributed messaging system such as Apache Kafka or MQTT-based infrastructure could collect events from traffic sensors, buses, cameras, and weather services. Kafka would distribute these events to multiple downstream applications.

Centralized storage would likely consist of a data lake or distributed object-storage system for historical data. Time-series databases such as InfluxDB or TimescaleDB could store sensor measurements, while spatial databases such as PostGIS could support location-based queries. Real-time engines such as Apache Flink or Spark Streaming could calculate traffic density, average vehicle speed, road occupancy, and public transport delays. Geospatial analytics could combine traffic coordinates with road-network information to identify congested areas.

For example:

Traffic Sensors → Kafka → Flink → Real-Time Congestion Detection → Alert/Dashboard
At the same time, historical data could be processed using Spark to discover long-term traffic patterns. Combining batch and streaming analytics allows city authorities to both respond to current congestion and plan future infrastructure.

Scalability would be achieved through distributed Kafka partitions, horizontally scalable processing clusters, replicated storage, and load balancing. Security would include encryption, authentication, role-based access control, network segmentation, and audit logging. Sensitive CCTV-related information should receive additional access restrictions.

Visualization systems such as Power BI, Tableau, Grafana, or custom GIS dashboards could display live traffic maps, congestion levels, public transport locations, weather conditions, and alerts. This provides city administrators with real-time operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

5. Healthcare Analytics and Predictive Health Platform

A healthcare analytics platform integrating patient records, wearable readings, laboratory reports, and appointment histories would most likely use a hybrid database and data-lake architecture.

Structured data such as patient records, laboratory values, prescriptions, and appointments could be stored in relational databases. Unstructured and semi-structured information such as medical documents, reports, device data, and clinical notes could be stored in distributed object storage or a healthcare data lake.

Healthcare interoperability would be supported using standards such as FHIR and HL7. ETL/ELT pipelines would extract information from hospitals and clinics, clean inconsistent values, transform records into standardized formats, and load them into the analytical platform.

Wearable devices could continuously send health measurements through an event-streaming platform such as Kafka. Flink or Spark Streaming could analyze the incoming data and detect abnormal values. For example, a sudden change in heart rate could trigger a monitoring alert.

For population-level analysis, Apache Spark would distribute processing across many machines. Instead of processing millions of patient records on a single server, the workload would be divided among multiple nodes. This enables large-scale calculations such as disease prevalence, readmission rates, treatment outcomes, and demographic health trends.

Data governance would be particularly important. The system would use:

- Encryption at rest and in transit
- Role-based access control
- Strong authentication
- Patient-data access auditing
- Data masking or de-identification for research
- Backup and disaster recovery
- Compliance monitoring
- Data retention policies

Machine learning could generate diagnosis risk scores, disease prediction, readmission-risk estimates, and treatment-support recommendations. Models would be trained using historical clinical and wearable data and evaluated before being used in clinical workflows.

The overall architecture can be represented as:

Steps in the Healthcare Analytics Pipeline

Step1: Data Collection

Patient records, wearable device readings, laboratory reports, and appointment histories are collected from different healthcare sources.

Step 2: Secure Data Ingestion

The collected data is transferred securely using ETL pipelines and real-time streaming

technologies.

Step 3: Data Storage

The data is stored in a **hybrid database and healthcare data lake**. Structured data is stored in databases, while semi-structured and unstructured data is stored in the data lake.

Step 4: Distributed Data Processing

Frameworks such as **Apache Spark and Apache Flink** process large volumes of healthcare data efficiently across multiple machines.

Step 5: Feature Engineering

Important features such as age, heart rate, laboratory values, previous diagnoses, and appointment patterns are extracted and prepared for machine learning.

Step 6: Machine Learning

Machine learning models analyze the prepared data to predict diseases, readmission risks, and other health outcomes.

Step 7: Risk Scores and Alerts

The system generates **risk scores, real-time alerts, and treatment recommendations** based on the model predictions.

Step 8: Dashboards and Decision Support

The results are displayed through **healthcare dashboards and clinical decision-support systems**, helping doctors and healthcare administrators make informed decisions.

Thus, the platform combines distributed storage, real-time streaming, batch analytics, machine learning, and strict security controls to support both individual patient care and population-level healthcare research.

